

Module 3: Variables

Introduction to C++ Programming — A Beginner's Course

Module Overview

Students already used variables informally in Module 2's exercises; this module makes the rules explicit (declaration vs. initialization, naming, scope, const) and introduces the two operators that make programs interactive: `std::cin` and `std::getline`. The BMI calculator project is the first program a student writes that behaves differently depending on what a real user types — a meaningful step up from Module 2's fixed-input converter.

Suggested Duration: 60–75 minutes

Learning Objectives

By the end of this module, students will be able to:

- Correctly declare, initialize, and reassign variables using appropriate naming conventions.
- Use compound assignment (`+=`, `-=`) and increment/decrement (`++`, `--`) operators.
- Explain what `const` enforces and why it's used for fixed values.
- Read user input with `std::cin >>` and `std::getline`, and explain the difference.
- Explain variable scope using nested `{ }` blocks.

Prerequisites

- Modules 1–2: comfortable with types, literals, and `std::cout`.

Materials & Instructor Prep

- 03-variables/examples/03_reading_input.cpp preloaded — this example needs live keyboard interaction, so run it, don't just show the code.

Key Vocabulary

Term	Definition
Declaration	Telling the compiler a variable exists and its type, e.g. <code>int age;</code>

Term	Definition
Initialization	Giving a variable its first value.
const	A qualifier that locks a variable's value — the compiler refuses to compile any later attempt to change it.
std::cin	The input stream representing keyboard input, paired with the extraction operator >>.
Scope	The region of code (typically a { } block) in which a variable exists and can be used.

Lesson Outline & Instructor Talking Points

1. Declaration, initialization, reassignment — three distinct ideas (15 min)

- Write `int age;` on the board and ask: 'what value does age hold right now?' Most will guess 0 — correct that it's actually undefined/garbage, and that reading it before assigning is a real bug source.
- Show the shorthand operators (`+=`, `++`) as convenience, not new concepts — they're just `score = score + 1` spelled differently. Have students translate a few by hand between the long and short forms.

Instructor note: *The 'uninitialized variable holds garbage, not zero' point is worth a full pause — it's counter to what many students assume from spreadsheet or math backgrounds.*

2. Naming: rules vs. conventions (10 min)

- Distinguish compiler-enforced rules (no spaces, can't start with a digit) from human convention (camelCase, descriptive names). Make clear the compiler will happily accept a name like `x7q` — the convention exists purely for the next human reading the code, which is often the same student a week later.

3. const — a safety net, demonstrated (10 min)

- Show `examples/02_constants.cpp`. Uncomment the illegal reassignment line live and let students see the compiler refuse to build.
- Frame `const` not as a restriction but as a guardrail: 'This isn't about being extra careful yourself — it's about the compiler catching a mistake made three files away in a large project, instantly, instead of the wrong answer only showing up in testing.'

4. Reading input: `cin >>` vs. `getline` (20 min)

- Live-run examples/03_reading_input.cpp. Deliberately type a full name with a space when prompted for the single-word input, and let the class watch it only capture the first word.
- Explain why: `std::cin >>` stops at whitespace. Introduce `std::getline` as the fix for full-line input, and explain the `std::cin.ignore()` quirk (leftover newline) as a known gotcha, not a mystery.

Instructor note: *This mismatch (`cin >>` stopping at a space) is one of the most common 'my program is silently wrong' bugs students hit on their own later. Naming it explicitly now saves debugging time in every subsequent module.*

5. Scope, revisited with nested blocks (10 min)

- Run examples/04_scope.cpp. Uncomment the out-of-scope access and let the compiler error make the rule concrete rather than abstract.
- Preview forward: 'This exact rule is why a loop variable you declare inside a loop's braces disappears the moment the loop ends — you'll see that constantly starting next module.'

Common Student Misconceptions

Misconception	How to Address It
"An uninitialized variable starts at 0."	It holds whatever garbage was already in that memory location — always initialize explicitly.
"= and == do the same thing."	= assigns a value; == compares two values. This distinction becomes safety-critical starting next module with if statements — plant the seed now.
" <code>std::cin >> name</code> reads a person's full name."	It stops at the first whitespace — demonstrate live, then introduce <code>getline</code> as the full-line alternative.

Exercises & Module Project

Exercise: 03-variables/exercises/exercise1.cpp — read name and age, compute years since a const legal age. Project: an interactive BMI calculator (03-variables/project/README.md) that reads real weight/height input — explicitly designed to be revisited and extended in Module 4.

Facilitation guidance: Tell students plainly that the BMI project is unfinished by design — it will grow an automatic category detector next module. This sets an expectation that programs evolve incrementally, mirroring the git-commit-often habit from Module 0.

Discussion & Comprehension Check Questions

1. What's the difference between `cin >> someString` and `getline(cin, someString)`?

Guidance: >> reads a single whitespace-delimited word; getline reads an entire line, spaces included.

2. Why might a real program want to prevent a variable from ever changing?

Guidance: Fixed facts (tax rates, physical constants, array sizes) — const catches accidental reassignment at compile time instead of producing a silently wrong result.

Assessment Ideas

- Trace-the-code exercise: give a 6-line snippet with `+=` and `++` operators and have students hand-compute the final value of a variable.

Answer Key Notes

See [03-variables/exercises/solutions/exercise1_solution.cpp](#).

Wrap-Up & Bridge to Next Module

Bridge forward: "Your BMI calculator can compute a number, but it can't yet tell the user what that number means — it can't make a decision. That's exactly what Module 4 adds."

Instructor Tips

- Have a working microphone/speaker moment where you type input live and get it wrong on purpose (e.g. typing a full name for the `>>` prompt) — a live mistake teaches this gotcha better than any slide.